FoveatedKV: Fused Importance-Adaptive KV Cache Compression with Spike-Driven Promotion for Apple Silicon

Anonymous

Abstract

We present FoveatedKV, a KV cache compression system for LLM inference on Apple Silicon that combines three ideas: (1) importance-adaptive two-tier quantization inspired by foveated rendering, keeping the top 10% of tokens at fp16 and compressing the rest with fp8 E4M3 keys and INT4 values; (2) a fused Split-K Metal compute kernel that reads quantized data directly from unified memory, dequantizes in registers, and performs spike detection as a free byproduct of online softmax; and (3) a C++ promotion pipeline that recovers exact fp16 representations from an NVMe-backed archive when the kernel detects a far-tier token receiving unexpectedly high attention. The asymmetric K/V precision strategy (higher precision for keys than values) builds on prior work by KIVI, LeanKV, and others. Our contribution is a complete system that implements this strategy within an importance-adaptive framework, targeting the memory-bandwidth constraints of Apple Silicon. On an 8GB Mac, FoveatedKV achieves 2.02× memory compression with 0.995+ cosine fidelity, makes a 7B model 2.3× faster than standard inference by reducing memory pressure, and recovers facts lost to quantization via spike-driven promotion.

1 Introduction

Large language model inference at long context is bottlenecked by memory bandwidth, not compute. Every decode step reads the entire KV cache from memory. For a 7B model at 32K context, the KV cache alone exceeds 1 GB. On consumer hardware with limited memory bandwidth (e.g., Apple M-series with 100–200 GB/s), this dominates inference latency.

Existing approaches to KV cache compression fall into two categories: *eviction* methods (H2O [1], SnapKV [2], StreamingLLM [3]) that discard low-importance tokens, and *quantization* methods (KIVI [4], LeanKV [5], KV-AdaQuant [6]) that reduce precision. Eviction methods break the softmax denominator by removing tokens from the sum. Uniform quantization treats high-attention tokens identically to filler text.

Recent work has established that keys and values have different quantization sensitivity: key errors are amplified through exp() in softmax, while value errors are bounded by attention weight. This observation has been independently reported by KIVI [4], LeanKV [5], KV-AdaQuant [6], and AsymKV [7], leading to proposals for asymmetric precision (e.g., K8V4). We build on this established insight.

Our contribution is *systems-level*: a complete implementation that combines asymmetric K/V quantization with importance-adaptive tiering, a fused GPU kernel, and a spike-driven promotion mechanism—all targeting the specific constraints of Apple Silicon. Concretely:

1. **Foveated two-tier cache.** Tokens are partitioned per attention head into a near tier (fp16) and far tier (fp8 K + INT4 V) based on attention importance. Every token remains in the softmax computation—no eviction, no broken denominators.

2. **Fused Split-K Metal kernel.** A single GPU dispatch reads mixed-precision data from unified memory, dequantizes in registers (fp8 via 256-entry LUT, INT4 via shift-and-mask), computes online softmax, and detects spikes—all without materializing fp16 intermediates to global memory.

3. **Spike detection and promotion.** The kernel identifies far-tier tokens receiving unexpectedly high attention scores as a free byproduct of online softmax tracking. A C++ background worker reads exact fp16 originals from an NVMe-backed mmap archive and promotes them into the near tier, recovering quality lost to quantization.

2 Background: Asymmetric K/V Precision

Multiple recent works have observed that keys and values have fundamentally different error sensitivity in the attention computation. We briefly review the theoretical basis, as it motivates our design choices.

Consider the attention output for a single query q:

$$\text{out} = \sum_i \frac{\exp(q \cdot k_i/\sqrt{d})}{\sum_j \exp(q \cdot k_j/\sqrt{d})} \cdot v_i = \sum_i \alpha_i v_i \tag{1}$$

Key error amplification. A quantization error ϵ_k on key k_i produces a score error $\delta_s = q \cdot \epsilon_k/\sqrt{d}$. Through exp(), this becomes a multiplicative factor on the attention weight:

$$\alpha_i' = \alpha_i \cdot \exp(\delta_s) \tag{2}$$

Because softmax normalizes, this error redistributes weight across *all* tokens in the sequence. KIVI [4] identified this through the lens of per-channel vs per-token quantization; LeanKV [5] proposed explicit K8V4 configurations; KV-AdaQuant [6] provided systematic analysis via singular value distributions and spectral norms; AsymKV [7] conducted a systematic examination of attention output error.

Value error is bounded. A quantization error ϵ_v on value v_i contributes $\alpha_i \cdot \epsilon_v$ to the output. For low-attention tokens, α_i is small, so the output error is bounded:

$$\|\text{error}\| \leq \sum_{i \in \text{far}} \alpha_i \|\epsilon_{v,i}\| \leq \alpha_{\text{far}} \cdot \max_i \|\epsilon_{v,i}\| \tag{3}$$

Our choice: fp8 E4M3 keys + INT4 values. Following LeanKV's K8V4 configuration, we use fp8 E4M3 for far-tier keys and INT4 for far-tier values. We prefer fp8 over INT8 for keys because the 256-entry lookup table (all fp8→fp16 mappings precomputed at kernel dispatch time) enables a single shared-memory read per element, replacing ∼10 ALU ops for per-channel INT8 dequantization. Our ablation (Table 8) confirms that fp8 and INT8 achieve equivalent error, while INT4 keys produce 34× more error.

We empirically validate the asymmetric choice on our system:

Configuration	Cosine vs Exact	Cosine Error
fp8 K + INT4 V (ours)	0.999939	0.000061
INT4 K + INT4 V (symmetric)	0.999783	0.000217

Table 1: Asymmetric vs symmetric K/V precision at 4K context. Symmetric INT4 produces 3.6× more attention error, consistent with findings in prior work [5, 6].

3 System Design

While the quantization strategy builds on prior work, the system architecture is our primary contribution: a foveated cache with a fused kernel and promotion pipeline, designed for Apple Silicon's unified memory.

3.1 Two-Tier Foveated Cache

Inspired by foveated rendering in VR [10]—where only the gaze center is rendered at full resolution—we partition tokens into two tiers per attention head after prefill:

- **Near tier** (default 10%): attention sinks (first N tokens) + recency window + top importance tokens, stored at fp16.

- **Far tier** (90%): remaining tokens, stored with fp8 E4M3 keys and INT4 (4-bit asymmetric per-token) values.

The "gaze" is the query vector; the "fovea" is the near tier; the "periphery" is the far tier. Unlike eviction methods, every token remains in the attention computation. The softmax denominator is exact. The only approximation is quantization noise on low-attention tokens.

This importance-adaptive approach differs from uniform quantization methods like KIVI [4]: high-attention tokens retain full precision, concentrating quantization noise where it has minimal impact. LeanKV [5] also combines importance-adaptive compression with asymmetric precision, but targets CUDA GPUs with a different kernel design. Our system targets Apple Silicon's unified memory architecture, which enables zero-copy access patterns that discrete-GPU systems cannot exploit.

3.2 Fused Split-K Metal Kernel

We implement a fused Metal compute kernel that performs the entire attention computation—across both tiers and the decode buffer—in a single GPU dispatch:

1. Load query, pre-scale by $1/\sqrt{d}$.
2. Near tier: standard fp16 dot product + online softmax accumulation.
3. Far tier: fp8 key dequantization via 256-entry threadgroup LUT (all 256 fp8→fp16 mappings precomputed at dispatch time—one shared-memory read replaces ∼10 ALU ops per element), dot product, score-gated INT4 value dequantization (skip when score $< m - 16$, i.e., $\exp(-16) \approx 10^{-7}$).
4. Decode buffer: new tokens added since compression (fp16).
5. Shared-memory reduction across Split-K groups.
6. Spike detection: track max(far scores) vs min(near scores) per head as a free byproduct of online softmax.

The kernel never materializes fp16 intermediates to global memory. Dequantization happens in registers. This is where the bandwidth savings come from: reading fp8+INT4 data costs 2–4× fewer bytes than fp16.

Key kernel optimizations: (a) pre-scaled query amortizes the $1/\sqrt{d}$ multiply across the entire sequence; (b) single-exp online softmax computes each exp() exactly once with FMA for rescale + accumulate; (c) 4-token blocked far loop provides instruction-level parallelism across independent K dot products; (d) score-gated V loading skips INT4 dequantization for negligible-attention tokens.

3.3 Spike Detection and Promotion

The spike detection mechanism is a distinguishing feature of our system. While LeanKV [5] and KIVI [4] apply static quantization, FoveatedKV can dynamically recover precision when needed.

During online softmax, the kernel naturally tracks max(far scores) and min(near scores) per head. When a far-tier token scores above the near-tier minimum, it is flagged as a *spike*. This detection adds zero computational overhead—the max/min values are already maintained by the softmax accumulator.

A C++ `PromotionPipeline` background worker handles promotion:

1. Reads the exact fp16 original from an NVMe-backed mmap archive (the archive preserves all pre-quantization KV pairs on disk).
2. Writes it into pre-allocated headroom slots in the near tier blob.
3. Atomically increments the per-head valid count.

The kernel sees the promoted token on the next dispatch as an ordinary near-tier token. Promotion is lossless—the disk archive preserves exact fp16 originals—and the unified memory architecture of Apple Silicon enables zero-copy access between the CPU promotion worker and the GPU kernel.

Filtering is critical. Raw spike rates are high (95% of head-layer slots per step), so the pipeline applies aggressive filtering: per-(layer,head) cooldown, splitmix64 position dedup, per-step budget cap, and GQA head dedup. Of 15,748 raw spikes over 80 decode steps, only 494 (3.1%) result in actual promotions. Without filtering, every far token would be promoted, defeating tiered compression.

4 Results

All experiments run on Apple M2 8GB. We evaluate on two model families: Qwen2.5 (0.5B quality, 7B throughput) and Llama-3.2-1B.

4.1 Kernel Performance

Fused kernel vs Apple's SDPA using 7B-equivalent tensor shapes (H_{kv}=8, H_q=32, D=128; these are the GQA dimensions of models like Qwen2.5-7B and Llama-3-8B), single layer:

Context	fp16 SDPA	Fused Kernel	Speedup
1K	1.12 ms	0.94 ms	1.19×
4K	2.33 ms	1.46 ms	1.60×
8K	4.05 ms	2.22 ms	1.82×
16K	7.68 ms	3.73 ms	2.06×
32K	15.72 ms	6.81 ms	2.31×

Table 2: Fused kernel speedup over Apple SDPA. Break-even at ∼512 tokens; bandwidth advantage grows with context length.

4.2 End-to-End: 7B on 8GB

On memory-constrained hardware, the compressed KV cache reduces memory pressure, making foveated *faster* than standard:

Context	Standard (tok/s)	Foveated (tok/s)	Speedup	Mem Saved
512	0.7	1.6	2.3×	—
1024	0.6	1.0	1.7×	50 MB
2048	0.2	0.4	2.0×	101 MB
3072	—	0.3	—	151 MB

Table 3: Qwen2.5-7B-Instruct-4bit on M2 8GB. Standard OOMs at ∼3–4K; foveated survives to 3K. Both are swap-bound; foveated wins by reading less memory.

Fused kernel correctness: cosine similarity vs dequant-reference = **1.000000** on both 7B (D=128) and Llama-1B (D=64) shapes.

4.3 Quality

To isolate kernel-level fidelity from model-specific effects, we measure attention output cosine similarity on synthetic random queries against real KV pairs from Qwen2.5-0.5B. KV tensors are shaped to match 7B GQA dimensions (H_{kv}=8, D=128) to verify that the kernel handles production-scale tensor shapes. This tests the quantization and kernel arithmetic, not end-to-end model quality (which is evaluated via perplexity in Table 5).

4.4 Perplexity

End-to-end perplexity on diverse multi-topic text:

Context	Cosine vs Exact	MAE	Compression
512	0.9956	0.0053	2.03×
1K	0.9950	0.0040	2.02×
4K	0.9952	0.0020	2.02×
8K	0.9954	0.0014	2.02×
16K	0.9953	0.0010	2.02×
32K	0.9954	0.0007	2.02×

Table 4: Kernel-level foveated attention fidelity at 10% near tier, measured on synthetic queries against real model KV pairs shaped to 7B GQA dimensions. Consistent 2.02× compression across all context lengths.

Model	Context	Standard PPL	Foveated PPL	Ratio
Qwen2.5-0.5B-4bit	1K	6.86	7.03	1.025×
Qwen2.5-0.5B-4bit	2K	15.14	15.12	0.999×
Qwen2.5-0.5B-4bit	4K	29.17	29.36	1.007×
Llama-3.2-1B-4bit	1K	8.12	8.11	0.999×
Llama-3.2-1B-4bit	2K	12.45	12.46	1.001×

Table 5: Perplexity within 2.5% of standard across both model families. Foveated compression does not measurably degrade language modeling quality. Note: these are 4-bit weight-quantized models, so absolute PPL is higher than fp16 baselines; the relevant metric is the foveated/standard *ratio*, which isolates the effect of KV cache compression from weight quantization.

4.5 LongBench-Lite

We evaluate on 6 representative LongBench [9] tasks (one per category, 10 samples each) using official THUDM scoring. Context is truncated to 2K tokens due to memory constraints on the 8GB test machine.

Task	Category	Standard	Foveated 10/90
qasper	Single-doc QA	2.0	1.6
hotpotqa	Multi-doc QA	0.0	0.0
qmsum	Summarization	5.3	7.0
triviaqa	Few-shot	1.0	1.0
passage_retrieval_en	Synthetic	0.0	0.0
lcc	Code	27.7	28.1
Average		**6.0**	**6.3**

Table 6: LongBench-Lite results (Qwen2.5-0.5B-Instruct-4bit, 2K context). Absolute scores are low due to the small 4-bit model and context truncation; the relevant comparison is foveated vs standard on the same model. Foveated matches or slightly exceeds standard on every task, confirming no quality degradation from KV cache compression.

4.6 Promotion Recovery

We evaluate the spike detection and promotion mechanism (Section 3.3) on a multi-fact retrieval benchmark: a biographical needle containing 4 checkable facts (dates, achievements) is buried in ∼2K tokens of filler text. With 5% near tier, the needle lands in the far tier. We compare three modes:

Without promotion, far-tier quantization degrades multi-token retrieval because the model must attend to the same facts across multiple decode steps. Spike detection identifies these tokens on the first decode step, promotion restores them to fp16, and subsequent steps benefit from the higher-quality representation.

Method	Facts Retrieved	Score
Standard (fp16 cache)	56/64	88%
Foveated + promotion	56/64	88%
Foveated, no promotion	32/64	50%

Table 7: Promotion recovers 24 additional facts (38% of total), matching the fp16 baseline. 100% reproducible across 8 seeds at depth=0.3. Model: Qwen2.5-0.5B-Instruct-4bit.

This mechanism has no analogue in prior asymmetric quantization work [4, 5, 6, 7], which apply static quantization without the ability to recover precision at inference time.

4.7 Key Precision Ablation

To validate our choice of fp8 E4M3 for keys (following LeanKV's [5] K8V4 recommendation), we measure single-layer attention cosine error at 4K context with different key formats (value format held constant at INT4):

Key Format	Cosine vs Exact	Cosine Error	vs fp8
fp16 (exact)	1.000000	0.000000	—
INT8 per-token	0.999991	0.000009	1.0×
fp8 E4M3 (ours)	0.999991	0.000009	1.0×
INT4 per-token	0.999683	0.000317	34.3×

Table 8: Key quantization format comparison at 4K context. fp8 E4M3 and INT8 are essentially equivalent; INT4 keys produce 34× more error. We prefer fp8 over INT8 because the 256-entry LUT enables a single shared-memory read per element vs per-channel scale/zero arithmetic for INT8.

4.8 Second Model Family: Llama-3.2-1B

To verify generalization beyond the Qwen architecture, we evaluate on Llama-3.2-1B-Instruct-4bit (GQA: H_q=32, H_{kv}=8, D=64):

- Fused kernel vs dequant-reference cosine: **1.000000** (kernel perfectly correct on Llama shapes).
- Foveated vs exact fp16 attention cosine: **0.994** (consistent with Qwen results at equivalent near%).
- Short-context token match: 100% (identical output to standard cache at 512 tokens).
- Throughput: 92.8 tok/s on M2 8GB.

4.9 Tier Ratio Ablation

We vary the near-tier percentage from 2% to 100% and measure single-layer attention cosine similarity vs exact fp16 at 2K context (Qwen2.5-0.5B-Instruct-4bit, layer 0):

The relationship between near percentage and cosine error is sub-linear: doubling the near tier from 10% to 20% only reduces error by 37% (0.0099 to 0.0062). This suggests that the most important tokens are captured even at small near percentages, with diminishing returns from adding more.

4.10 Spike Detection Analysis

We instrument the fused kernel to measure raw spike frequency during generation (2K context, 5% near tier, 80 decode steps).

Near %	Near Tokens	Far Tokens	Cosine	Error
2%	40	2008	0.9766	0.0234
5%	102	1946	0.9844	0.0156
10%	204	1844	0.9901	0.0099
15%	307	1741	0.9913	0.0087
20%	409	1639	0.9938	0.0062
30%	614	1434	0.9952	0.0048
50%	1024	1024	0.9970	0.0030
75%	1536	512	0.9982	0.0018
100%	2048	0	1.0000	0.0000

Table 9: Attention fidelity vs near-tier percentage at 2K context. The default 10% achieves 0.990 cosine with 2× compression. Quality degrades gracefully: even at 2% near, cosine remains above 0.976.

Frequency. The kernel detects spikes on 100% of decode steps, averaging 182 raw spikes per step across 24 layers × 8 heads (192 head-layer slots). This corresponds to a 95% spike rate per head-layer slot.

Effective rate. Of 15,748 raw spikes over 80 steps, only 494 (3.1%) result in actual promotions after the filtering pipeline. The 32× reduction from raw to effective demonstrates that naive promotion would defeat tiered compression.

Layer distribution. Spikes are distributed roughly uniformly across layers (4–6% each), with slightly higher rates in middle layers (layer 16: 6.1%) and the final layer (layer 23: 5.8%). No single layer dominates, confirming that per-layer independent spike detection is the right design.

5 Related Work

KV cache eviction. H2O [1] uses cumulative attention scores to identify heavy hitters and prunes the rest. StreamingLLM [3] keeps attention sinks and a sliding window. SnapKV [2] selects important tokens based on observation-window attention patterns. All three remove tokens from the softmax sum, changing the function being computed. FoveatedKV retains all tokens.

Asymmetric K/V quantization. KIVI [4] first demonstrated that keys and values require different quantization strategies (per-channel for keys, per-token for values) and proposed tuning-free INT2/INT4 quantization. LeanKV [5] made the bit-width asymmetry explicit, proposing K8V4 configurations and combining them with importance-adaptive token-level compression and per-head dynamic sparsity. KV-AdaQuant [6] provided systematic theoretical justification through singular value and spectral norm analysis, proposing K4V2 allocations. AsymKV [7] from Alibaba conducted systematic examination of attention output error and proposed distinct configurations for key and value matrices. KVSplit [8] implements K8V4 differentiated precision specifically for Apple Silicon with Metal support, making it the closest existing system to FoveatedKV on the same hardware target. KVSplit applies uniform K8V4 quantization to all tokens; FoveatedKV differs in three ways: (a) importance-adaptive tiering (the top 10% of tokens retain fp16, concentrating quantization noise on low-attention tokens rather than applying it uniformly), (b) a fused kernel that computes attention across both tiers in a single dispatch with integrated spike detection, and (c) the promotion pipeline that dynamically recovers full-precision representations from disk when the model's attention pattern shifts. The tiering and promotion are complementary to the asymmetric precision that KVSplit and FoveatedKV share.

FoveatedKV adopts the established fp8/INT4 asymmetric precision and contributes: (a) the foveated two-tier architecture with per-head importance-adaptive partitioning, (b) the fused Metal kernel with integrated spike detection at zero overhead, and (c) the promotion pipeline that dynamically recovers fp16 precision from NVMe-backed archives—a capability not present in any of the above systems.

Foveated rendering. Foveated rendering in VR [10] renders the gaze center at full resolution and reduces quality in the periphery. We adapt this principle to attention: the "gaze" is the query vector, the "fovea" is the near tier, and the "periphery" is the far tier.

6 Limitations

Model scale. Quality results are on Qwen2.5-0.5B and Llama-3.2-1B; throughput results on Qwen2.5-7B. Both Qwen (GQA 8-head, D=128) and Llama (GQA 8-head, D=64) architectures show consistent behavior. We have not evaluated on 13B+ models. Prior work on asymmetric K/V quantization [5, 6] has validated the approach at larger scales; our system-level contributions (kernel, promotion) need separate validation.

Evaluation breadth. End-to-end quality is evaluated on LongBench-Lite (Table 6), perplexity, and a custom multi-fact retrieval benchmark. LongBench results are on a 0.5B model with 2K context truncation; full-length evaluation on larger models with RULER or complete LongBench would strengthen the claims.

Hardware. All results are on Apple M2 8GB. The 7B throughput numbers are swap-bound and reflect memory-pressure dynamics specific to this hardware. On machines with more memory (16–32GB), the standard baseline would be faster, changing the crossover point. The kernel bandwidth advantage (2.3× at 32K) is hardware-independent.

Comparison scope. We do not directly benchmark against LeanKV [5] or KV-AdaQuant [6]. Kernel-level comparison is infeasible (those systems target CUDA), but the algorithmic components—tier assignment policy, compression quality at equivalent bit-budgets—could be compared via Python reference implementations on the same models and benchmarks. A controlled algorithmic comparison, and a head-to-head with llama.cpp's quantized KV cache on the same Mac hardware, would meaningfully strengthen the evaluation. We leave these for future work.

7 Conclusion

FoveatedKV demonstrates that combining established asymmetric K/V quantization with importance-adaptive tiering, a fused Metal kernel, and spike-driven promotion yields a practical system for memory-constrained Apple Silicon inference. The system achieves 2× memory compression with negligible quality loss. On an 8GB Mac, it is faster than standard inference because reading less memory dominates. The promotion mechanism—detecting spikes during online softmax at zero overhead and recovering exact fp16 from NVMe-backed archives—closes the quality gap for sustained multi-fact retrieval, a capability not present in prior quantization-only approaches.

References

[1] Zhang et al., "H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models," NeurIPS 2023.

[2] Li et al., "SnapKV: LLM Knows What You Are Looking For Before Generation," 2024.

[3] Xiao et al., "Efficient Streaming Language Models with Attention Sinks," ICLR 2024.

[4] Liu et al., "KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache," ICML 2024.

[5] Chen et al., "LeanKV: KV Cache Compression with Differentiated Key-Value Precision," arXiv:2412.03131, 2024.

[6] Li et al., "KV-AdaQuant: Adaptive KV Cache Quantization with Key-Value Precision Allocation," arXiv:2502.15075, 2025.

[7] Li et al., "AsymKV: Asymmetric Key-Value Cache Quantization," arXiv:2410.13212, 2024.

[8] Paul, "KVSplit: Differentiated Key-Value Precision for Apple Silicon," `https://github.com/dipampaul17/KVSplit`, 2025.

[9] Bai et al., "LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding," ACL 2024.

[10] Guenter et al., "Foveated 3D Graphics," ACM SIGGRAPH Asia 2012.